# INDEX

| Sr.No. | Practicals | Date | Sign |
|---|---|---|---|
| 1 | Implementing advanced deep learning algorithms such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs) using Python libraries like TensorFlow or PyTorch. | | |
| 2 | Building a natural language processing (NLP) model for sentiment analysis or text classification. | | |
| 3 | Creating a chatbot using advanced techniques like transformer models. | | |
| 4 | Developing a recommendation system using collaborative filtering or deep learning approaches. | | |
| 5 | Implementing a computer vision project, such as object detection or image segmentation. | | |
| 6 | Applying reinforcement learning algorithms to solve complex decision-making problems. | | |
| 7 | Building a deep learning model for time series forecasting or anomaly detection. | | |
| 8 | Implementing a machine learning pipeline for automated feature engineering and model selection. | | |
| 9 | Using advanced optimization techniques like evolutionary algorithms or Bayesian optimization for hyperparameter tuning. | | |
| 10 | Deploying a machine learning model in a production environment using containerization and cloud services. | | |

# PRACTICAL NO – 1

**Aim - Implementing advanced deep learning algorithms such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs) using Python libraries like TensorFlow or PyTorch.**

**Code-**

```
# Import necessary libraries
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
# Load and preprocess the MNIST dataset

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape((x_train.shape[0], 28, 28, 1)).astype("float32") / 255.0

x_test = x_test.reshape((x_test.shape[0], 28, 28, 1)).astype("float32") / 255.0

# One-hot encode the labels

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
# Build the CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy'])
# Train the model

history = model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.2)

# Evaluate the model on the test set

test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
# Plot training and validation accuracy and loss
plt.figure(figsize=(12, 4))
```

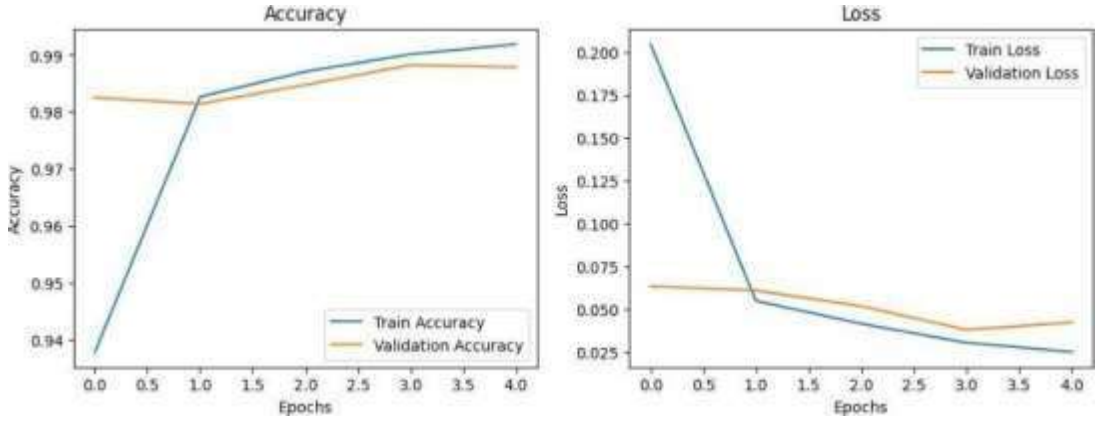
```
plt.subplot(1, 2, 1)

plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1, 2, 2)

plt.plot(history.history['loss'], label='Train Loss')


plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

## OUTPUT:-



# PRACTICAL NO – 2

**Aim - Building a natural language processing (NLP) model for sentiment analysis or text classification.**

## Code-

```
import tensorflow as tf

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import reuters
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as plt
# Custom callback for cleaner logs

class ClearLogsCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        print(f"Epoch {epoch + 1}: Accuracy = {logs['accuracy']:.4f}, Loss = {logs['loss']:.4f}")

# Load the Reuters dataset

vocab_size = 10000  # Top 10,000 words
maxlen = 200 # Maximum sequence length
(train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=vocab_size)
# Pad sequences to ensure uniform input length
train_data = pad_sequences(train_data, maxlen=maxlen)
test_data = pad_sequences(test_data, maxlen=maxlen)
# Convert labels to one-hot encoding
num_classes = max(train_labels) + 1
train_labels = tf.keras.utils.to_categorical(train_labels, num_classes)

test_labels = tf.keras.utils.to_categorical(test_labels, num_classes)

# Build the model

model = models.Sequential()

# Add an embedding layer

model.add(layers.Embedding(input_dim=vocab_size, output_dim=128))

# Add an LSTM layer

model.add(layers.LSTM(64))
# Add dense layers for classification
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(num_classes, activation='softmax'))  # Multi-class classification

# Compile the model
        model.compile(optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy'])
    # Train the model with custom callback
```

```
    history = model.fit(

    train_data,
    train_labels,
    epochs=5,
    batch_size=64,
    validation_data=(test_data, test_labels),
    verbose=0, # Suppress dynamic progress bar
    callbacks=[ClearLogsCallback()]  # Use custom callback for logs

    )

# Evaluate the model on the test data

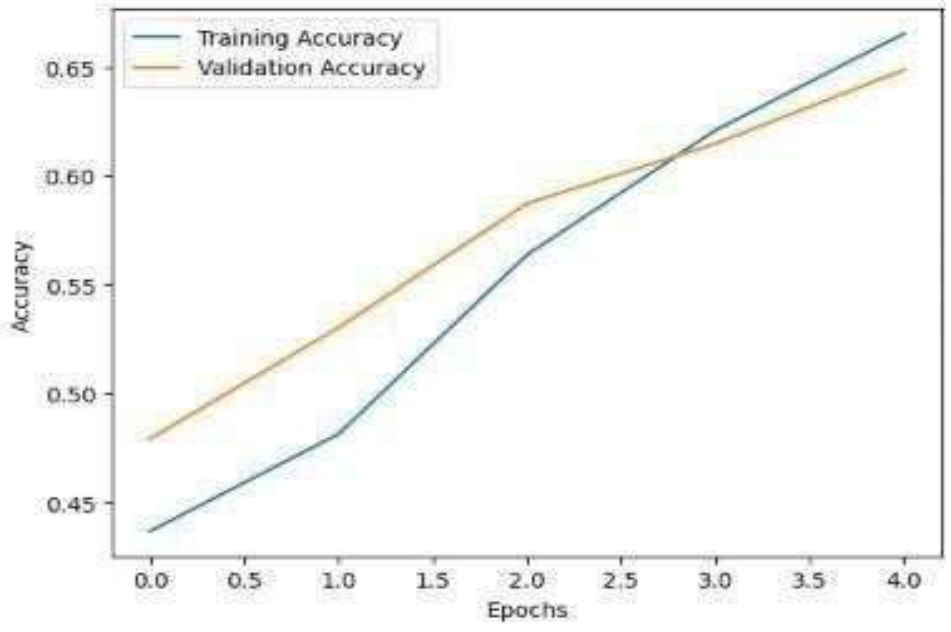
test_loss, test_acc = model.evaluate(test_data, test_labels, verbose=0)
print(f"\nTest accuracy: {test_acc}")

# Plot training and validation accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

## OUTPUT –

```
Epoch 1: Accuracy = 0.4367, Loss = 2.3768
Epoch 2: Accuracy = 0.4813, Loss = 2.0012
Epoch 3: Accuracy = 0.5637, Loss = 1.6856
Epoch 4: Accuracy = 0.6210, Loss = 1.4686
Epoch 5: Accuracy = 0.6654, Loss = 1.2886

Test accuracy: 0.6487088203430176
```



# PRACTICAL NO – 3

**Aim - Creating a chatbot using advanced techniques like transformer models.**
**Code-**

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import os

# Suppress TensorFlow logs
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
# Suppress Hugging Face symlink warnings
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
def chatbot():

    # Load pre-trained GPT-2 model and tokenizer
    print("Loading model...")
    model_name = "gpt2"  # You can replace this with 'gpt2-medium' or another variant

    # Load the tokenizer and model

    tokenizer = GPT2Tokenizer.from_pretrained(model_name)
    model = GPT2LMHeadModel.from_pretrained(model_name)
    # Set a padding token (GPT-2 doesn't have one by default)
    tokenizer.pad_token = tokenizer.eos_token
    print("Model loaded successfully!")

    # Instructions


    print("\nChatbot ready! Type 'exit' to quit.\n")

    # Chat loop
    while True:
        user_input = input("You: ")

        if user_input.lower() == "exit":
            print("Chatbot:  Goodbye!")
            break


        # Tokenize and generate a response with attention mask

        inputs = tokenizer(user_input, return_tensors="pt", padding=True, truncation=True)
        input_ids = inputs["input_ids"]
        attention_mask = inputs["attention_mask"]

         # Generate the response
         output = model.generate(
            input_ids,

            attention_mask=attention_mask, # Include attention mask
            max_length=100,
            num_return_sequences=1,
```

```
        pad_token_id=tokenizer.pad_token_id  # Use explicitly defined padding token
    )

    # Decode and print the response
    response = tokenizer.decode(output[0], skip_special_tokens=True)
    print(f"Chatbot: {response[len(user_input):].strip()}") # Remove user's input from
    response

# Run the chatbot

if __name__ == "__main__":
chatbot()
```

**OUTPUT –**

```
You: Hello
Chatbot: , I'm sorry, but I'm not sure if
You: How are you ?
Chatbot: I am a professional photographer.
You: Exit
Chatbot: Goodbye!
```

# PRACTICAL NO – 4

**Aim - Developing a recommendation system using collaborative filtering or deep learning approaches.**

**Code-**

```
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, Flatten, Dense, Concatenate
from sklearn.model_selection import train_test_split
# Sample data (user, item, rating)
data = np.array([
    [1, 101, 5], [1, 102, 3], [2, 101, 4], [2, 103, 2],
    [3, 102, 4], [3, 104, 1]
])
users = data[:, 0]
items = data[:, 1]
ratings = data[:, 2]
# Normalize ratings
ratings = (ratings - ratings.min()) / (ratings.max() - ratings.min())
# Train-test split
train_users, test_users, train_items, test_items, train_ratings, test_ratings = train_test_split(
    users, items, ratings, test_size=0.2, random_state=42
)
# Number of unique users and items
num_users = len(np.unique(users))
num_items = len(np.unique(items))

# Model architecture
user_input = Input(shape=(1,))
item_input = Input(shape=(1,))
```

```
    user_embedding = Embedding(num_users + 1, 50)(user_input)
    item_embedding = Embedding(num_items + 1, 50)(item_input)
    user_vec = Flatten()(user_embedding)
    item_vec = Flatten()(item_embedding)
concat = Concatenate()([user_vec, item_vec])
dense = Dense(128, activation='relu')(concat)
output = Dense(1, activation='sigmoid')(dense)
model = Model(inputs=[user_input, item_input], outputs=output)
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
# Train the model
model.fit([train_users, train_items], train_ratings, epochs=10, batch_size=16, verbose=1)
# Test the model
predicted_ratings = model.predict([test_users, test_items])
print(f"Predicted ratings: {predicted_ratings.flatten()}")
```

## OUTPUT –

```
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step - loss: 0.1080 - mae: 0.3105
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step - loss: 0.1054 - mae: 0.3071
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.1030 - mae: 0.3038
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 0.1009 - mae: 0.3008
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step - loss: 0.0988 - mae: 0.2977
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step - loss: 0.0967 - mae: 0.2947
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - loss: 0.0946 - mae: 0.2916
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0925 - mae: 0.2885
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step - loss: 0.0904 - mae: 0.2853
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step - loss: 0.0883 - mae: 0.2820
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 217ms/step
Predicted ratings: [0.5275267  0.51566106]
```

# PRACTICAL NO – 5

**Aim - Implementing a computer vision project, such as object detection or image segmentation.**

**Code-**

```
# Install required libraries

# pip install ultralytics requests

from ultralytics import YOLO

import cv2

import numpy as np

import requests

from matplotlib import pyplot as plt

# Load the pre-trained YOLO model

model = YOLO('yolov8n.pt')  # Replace with the appropriate model version

# Download the image from a URL

image_url = 'https://i.sstatic.net/UYYqo.jpg'
response = requests.get(image_url, stream=True)

image_array = np.asarray(bytearray(response.content), dtype=np.uint8)

image = cv2.imdecode(image_array, cv2.IMREAD_COLOR)

# Convert BGR to RGB for visualization

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Perform object detection

results = model(image)

# Show the detected results

results_image = results[0].plot()  # Annotated image with bounding boxes and labels


# Display the output using matplotlib

plt.figure(figsize=(10, 10))

plt.imshow(cv2.cvtColor(results_image, cv2.COLOR_BGR2RGB))

plt.axis('off')

plt.show()
```

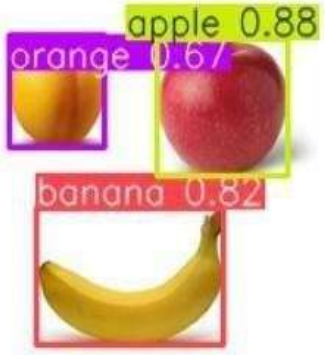
# Print detected objects and their confidence scores

for result in results[0].boxes:

print(f"Object: {result.data[0][5]}, Confidence: {result.data[0][4]:.2f}")

## OUTPUT –

```
0: 640x640 1 banana, 1 apple, 1 orange, 998.9ms
Speed: 20.4ms preprocess, 998.9ms inference, 2.2ms postprocess per image at shape (1, 3, 640, 640)
```




```
Object: 47.0, Confidence: 0.88
Object: 46.0, Confidence: 0.82
Object: 49.0, Confidence: 0.67
```

# RACTICAL NO – 6

**Aim - Applying reinforcement learning algorithms to solve complex decision-making problems.**

**Code-**

```
import numpy as np

import random

# Define the environment

grid_size = 5

goal = (4, 4)

obstacles = [(1, 1), (2, 3), (3, 1)]

# Q-learning parameters

alpha = 0.1  # Learning rate

gamma = 0.9  # Discount factor

epsilon = 0.1 # Exploration rate

episodes = 1000

# Initialize the Q-table

q_table = np.zeros((grid_size, grid_size, 4)) # 4 actions: up, down, left, right

# Define actions

actions = {

    0: (-1, 0),  # Up

    1: (1, 0),   # Down

    2: (0, -1),  # Left

    3: (0, 1)    # Right

}

def is_valid_position(pos):

    """Check if the position is valid (not out of bounds or in an obstacle)."""

    x, y = pos

    if x < 0 or x >= grid_size or y < 0 or y >= grid_size:
```

```
        return False
    if pos in obstacles:
        return False
    return True
def get_next_state(state, action):
    """Get the next state after taking an action."""
    x, y = state
    dx, dy = actions[action]
    next_state = (x + dx, y + dy)
    if is_valid_position(next_state):
        return next_state
    return state # Stay in the same position if invalid move
def get_reward(state):
    """Return the reward for a given state."""
    if state == goal:
        return 10
    if state in obstacles:
        return -10
    return -1
# Training the agent
for episode in range(episodes):
    state = (0, 0) # Start position
    total_reward = 0
    while state != goal:
        # Choose an action (epsilon-greedy strategy)
        if random.uniform(0, 1) < epsilon:
            action = random.choice(list(actions.keys())) # Explore
        else:
            action = np.argmax(q_table[state[0], state[1]]) # Exploit
```

```
        # Take the action
        next_state = get_next_state(state, action)
        reward = get_reward(next_state)
        # Update Q-value using the Q-learning formula
        old_value = q_table[state[0], state[1], action]
        next_max = np.max(q_table[next_state[0], next_state[1]])
        q_table[state[0], state[1], action] = old_value + alpha * (reward + gamma * next_max - old_value)
        state = next_state
        total_reward += reward
    if (episode + 1) % 100 == 0:
        print(f"Episode {episode + 1}, Total Reward: {total_reward}")
# Test the agent
state = (0, 0)
path = [state]
while state != goal:
    action = np.argmax(q_table[state[0], state[1]])
    state = get_next_state(state, action)
    path.append(state)
print("Optimal Path:", path)
```

## Output –

```
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug  6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/Guest Login/Documents/advanced ai/REINFORCEMENT LEARNING ALGORITHM.py
Episode 100, Total Reward: 3
Episode 200, Total Reward: 1
Episode 300, Total Reward: 3
Episode 400, Total Reward: 3
Episode 500, Total Reward: 2
Episode 600, Total Reward: 1
Episode 700, Total Reward: 3
Episode 800, Total Reward: 1
Episode 900, Total Reward: 3
Episode 1000, Total Reward: 0
Optimal Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (4, 2), (4, 3), (4, 4)]
```

# PRACTICAL NO – 7

**Aim - Building a deep learning model for time series forecasting or anomaly detection**

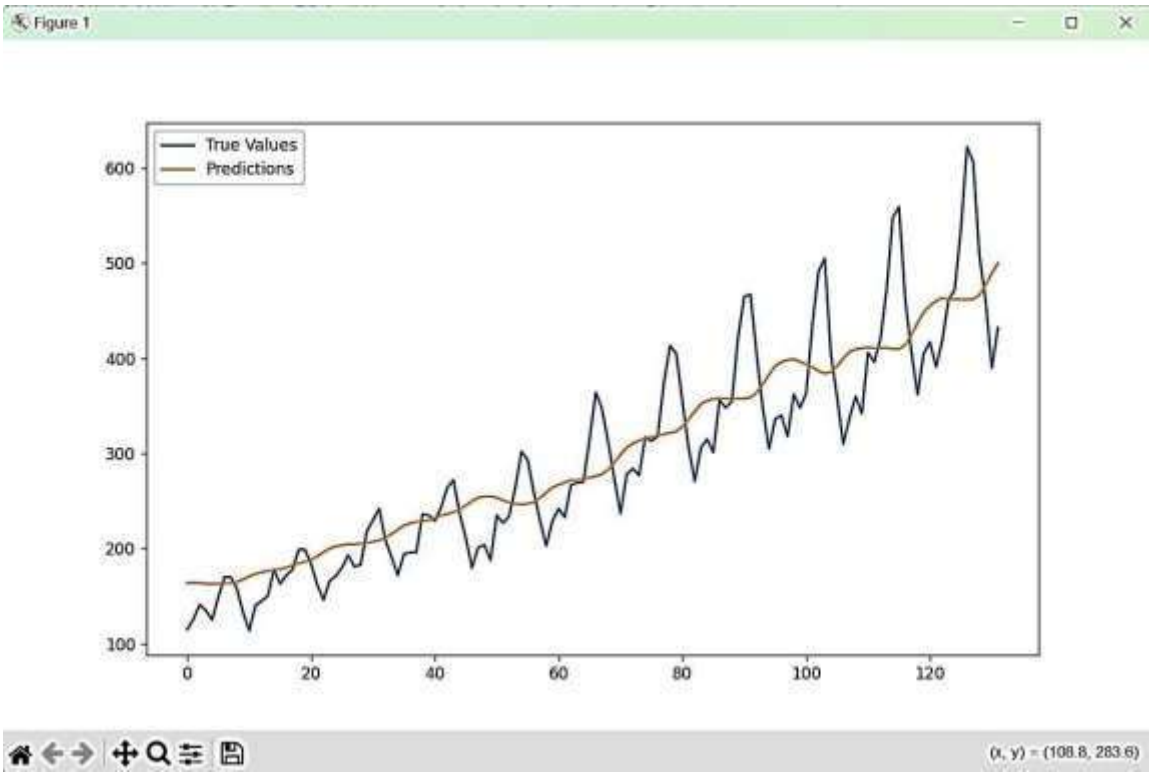
**Code-**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
# Step 1: Load dataset (using a new example dataset)
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv'
data = pd.read_csv(url, header=0, index_col=0, parse_dates=True)
# Show the first few rows of the data
print(data.head())
# Step 2: Preprocessing - Using the dataset's values
data = data.values
data = data.reshape(-1, 1)
# Normalize the data using MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)
# Step 3: Create sequences of data for training
def create_sequences(data, time_step=12):
    X, y = [], []
    for i in range(len(data) - time_step):
        X.append(data[i:i + time_step])
        y.append(data[i + time_step])
    return np.array(X), np.array(y)
```

```
time_step = 12  # Using 12 previous months to predict the next one
X, y = create_sequences(scaled_data, time_step)
# Reshape the data for LSTM (samples, time steps, features)
X = X.reshape(X.shape[0], X.shape[1], 1)
# Step 4: Build the LSTM model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X.shape[1], 1)))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dense(units=1))
# Compile the model
model.compile(optimizer=Adam(learning_rate=0.001),  loss='mean_squared_error')
# Step 5: Train the model
model.fit(X, y, epochs=10, batch_size=32)
# Step 6: Make predictions
predictions = model.predict(X)
# Step 7: Inverse transform the predictions
predictions = scaler.inverse_transform(predictions)
# Plot the predictions and the true values
plt.figure(figsize=(10, 6)) plt.plot(data[time_step:],
label="True Values") plt.plot(predictions,
label="Predictions") plt.legend()
plt.show()
```

**OUTPUT-**

```
========== RESTART: C:/Users/
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
```


Figure 1
True Values
Predictions
600
500
400
300
200
100
0
20
40
60
80
100
120
(x, y) = (108.8, 283.6)

# PRACTICAL NO – 8

**Aim - Implementing a machine learning pipeline for automated feature engineering and model selection**

**Code-**

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from tpot import TPOTClassifier
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

# Step 1: Load and Prepare Data

# Here we use the Iris dataset as an example
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)
# Step 2: Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 3: Preprocessing and Feature Engineering
# Define which columns are numeric
numeric_features = X.columns.tolist()
# We can apply different transformations to numeric features
numeric_transformer = Pipeline(steps=

        ('imputer', SimpleImputer(strategy='mean')), # Handle missing values
        ('scaler', StandardScaler())  # Normalize the data
])

# Combine everything into a preprocessor
preprocessor = ColumnTransformer(
    transformers=[

        ('num', numeric_transformer, numeric_features)

    ]

    )
```

```
# Step 4: Define a Model Selection Pipeline using TPOT

# Use TPOT to automatically select the best model and parameters

tpot = TPOTClassifier( generations=5, population_size=20, random_state=42, verbosity=2)

# Step 5: Build the full pipeline
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),  # Apply preprocessing
    ('model', tpot) # Automated model selection with TPOT
])

# Step 6: Train the model using the training data
pipeline.fit(X_train, y_train)
# Step 7: Evaluate the model on the test set
y_pred = pipeline.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')

# Optional: Export the best model pipeline found by TPOT
tpot.export('best_model_pipeline.py')
```

## OUTPUT-



# PRACTICAL NO – 9

**Aim – Using advanced optimization techniques like evolutionary algorithms or Bayesian optimization for hyperparameter tuning.**

## 1. Bayesian Optimization with Optuna

**Code-**

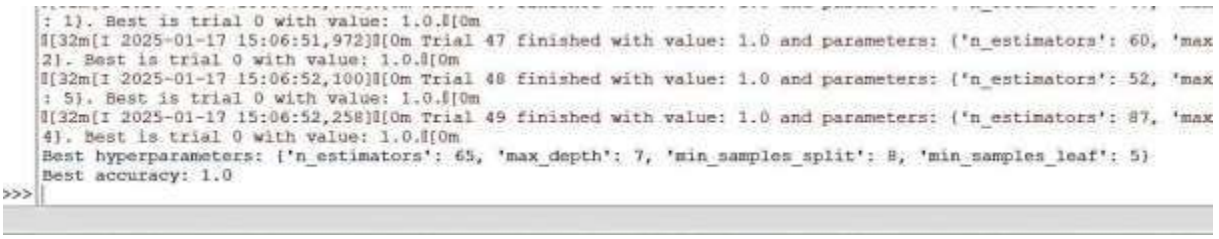
```
import optuna
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
# Load the Iris dataset
data = load_iris()
X = data.data
y = data.target
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# Define an objective function for Optuna optimization
def objective(trial):
    # Define hyperparameters to tune
    n_estimators = trial.suggest_int('n_estimators', 10, 100)
    max_depth = trial.suggest_int('max_depth', 3, 10)
    min_samples_split = trial.suggest_int('min_samples_split', 2, 10)
    min_samples_leaf = trial.suggest_int('min_samples_leaf', 1, 5)
    # Create a Random Forest Classifier model
    model = RandomForestClassifier(n_estimators=n_estimators,
                                   max_depth=max_depth,
                                   min_samples_split=min_samples_split,
                                   min_samples_leaf=min_samples_leaf,
                                   random_state=42)
        # Train the model
        model.fit(X_train, y_train)
        # Evaluate the model
y_pred = model.predict(X_test) return
accuracy_score(y_test, y_pred)

# Set up the Optuna study for optimization
study = optuna.create_study(direction="maximize")

# Optimize hyperparameters
study.optimize(objective, n_trials=50)

# Print the best hyperparameters found by Optuna
print(f"Best hyperparameters: {study.best_params}")
print(f"Best accuracy: {study.best_value}")
```

**OUTPUT -**




## 2. Evolutionary Algorithms with TPOT

**Code:-**

```
from tpot import TPOTClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
  # Load the Iris dataset
  data = load_iris()
X = data.data
y = data.target
# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


  # Initialize the TPOTClassifier (using evolutionary algorithms for hyperparameter optimization)

tpot = TPOTClassifier( generations=5, population_size=20, random_state=42, verbosity=2)


# Train the model and optimize hyperparameters
tpot.fit(X_train, y_train)


# Evaluate the model on the test set
y_pred = tpot.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')


# Export the best model pipeline found by TPOT
tpot.export('best_model_pipeline.py')
```

# OUTPUT-




```
[I 2025-01-26 08:40:07,557] Trial 47 finished with value: 1.0 and parameters: {'n_estimators': 128, 'max_depth
[I 2025-01-26 08:40:07,629] Trial 48 finished with value: 1.0 and parameters: {'n_estimators': 27, 'max_depth'
[I 2025-01-26 08:40:07,765] Trial 49 finished with value: 1.0 and parameters: {'n_estimators': 66, 'max_depth'
Best Hyperparameters: {'n_estimators': 182, 'max_depth': 48, 'min_samples_split': 10, 'min_samples_leaf': 7}
Best Accuracy: 1.0000
```

# PRACTICAL NO-10

## Aim:Deploying a machine learning model in a production environment using containerization and cloud services.

## Code:-

```
!pip install transformers datasets

from transformers import GPT2LMHeadModel, GPT2Tokenizer, Trainer,
TrainingArguments
from datasets import load_dataset

# Load a pre-trained GPT-2 model and tokenizer model_name =
"gpt2"
model = GPT2LMHeadModel.from_pretrained(model_name)
tokenizer=GPT2Tokenizer.from_pretrained(model_name)

# Define the padding token (usually the EOS token for GPT-2) tokenizer.pad_token =
tokenizer.eos_token # This line is crucial to fix the error

# Load a text dataset (You can replace this with your own dataset) dataset =
load_dataset("wikitext", "wikitext-103-raw-v1")

# Preprocess the dataset (Tokenization) def
encode(example):
    return tokenizer(example['text'], truncation=True, padding="max_length",
max_length=512)

train_data = dataset['train'].map(encode, batched=True) val_data =
dataset['validation'].map(encode, batched=True)

# Set up training arguments training_args =
TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    logging_dir="./logs", logging_steps=10,
    save_steps=10, warmup_steps=100,
    weight_decay=0.01,
    evaluation_strategy="steps",
    save_total_limit=2,
)

# Set up Trainer trainer =
Trainer(
    model=model,
```

```
        train_dataset=train_data,
        eval_dataset=val_data,
)

# Train the model trainer.train()
```

args=training_args,

OUTPUT-

